KMP Algorithm

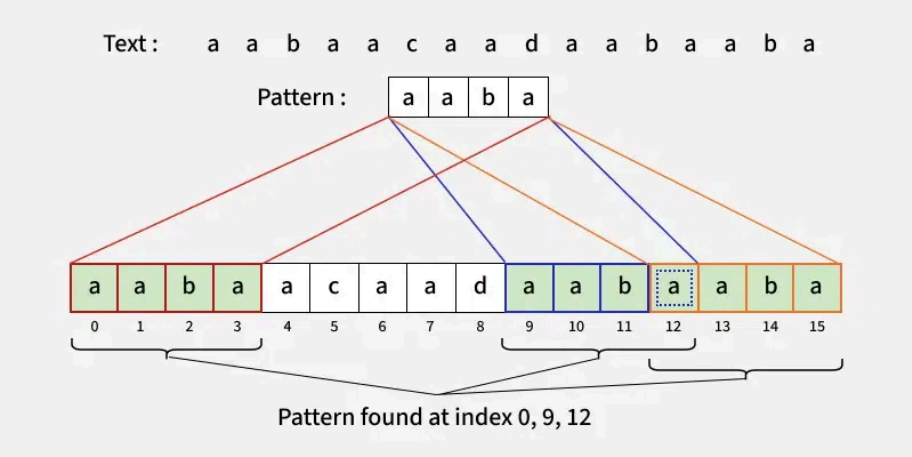
Last Updated : 27 Mar, 2026

Given two strings **txt** and **pat, t**he task is to return all indices of **occurrences** of **pat** within **txt**.

Examples:

Input: *txt = "abcab", pat = "ab"*
Output: *[0, 3]*
Explanation*: The string "ab" occurs twice in txt, first occurrence starts from index 0 and second from index 3.*

Input: *txt= "aabaacaadaabaaba", pat = "aaba"*
Output: *[0, 9, 12]*
Explanation:



Naive Algorithm

- We start at every index in the text and compare it with the first character of the pattern, if they match we move to the next character in both text and pattern.
- If there is a mismatch, we start the same process for the next index of the text.

Please refer Naive algorithm for pattern searching for implementation.

KMP Algorithm

The Naive Algorithm can work in linear time if we know for sure that all characters are distinct. Please refer Naive Pattern Searching for Distinct Characters in Pattern. The Naive algorithm can not be made better than linear when we have repeating characters.

> *Examples of patterns with repeating subpatterns*
> ***1)*** *txt[] = "AAAAAAAAAAAAAAAAAB", pat[] = "AAAAB"*
> ***2)*** *txt[] = "ABABABCABABABCABABABC", pat[] = "ABABAC" (not a worst case, but a bad case for Naive)*

The KMP matching algorithm uses degenerating property (pattern having the same sub-patterns appearing more than once in the pattern) of the pattern and improves the worst-case complexity to **O(n+m)**.

The basic idea behind KMP's algorithm is: whenever we detect a mismatch (after some matches), we already know some of the characters in the text of the next window. We take advantage of this information to avoid matching the characters that we know will anyway match.

Matching Overview

txt = "AAAAABAAABA"
pat = "AAAA"
We compare first window of ***txt*** *with* ***pat***

*txt = "****AAAA****ABAAABA"*
*pat = "****AAAA****" [Initial position]*
We find a match. This is same as Naive String Matching.

In the next step, we compare next window of ***txt*** *with* ***pat****.*

*txt = "****AAAAA****BAAABA"*
*pat = "****AAAA****" [Pattern shifted one position]*

This is where KMP does optimization over Naive. In this second window, we only compare fourth A of pattern with fourth character of current window of text to decide whether current window matches or not. Since we know first three characters will anyway match, we skipped matching first three characters.

Need of Preprocessing?

An important question arises from the above explanation, how to know how many characters to be skipped. To know this,
we pre-process pattern and prepare an integer array lps[] that tells us the count of characters to be skipped

In KMP Algorithm,

- We preprocess the pattern and build LPS array for it. The size of this array is same as pattern length.
- LPS is the **Longest Proper Prefix** which is also a **Suffix**. A proper prefix is a prefix that doesn't include **whole** string. For example, prefixes of "abc" are "", "a", "ab" and "abc" but proper prefixes are "", "a" and "ab" only. Suffixes of the string are "", "c", "bc", and "abc".
- Each value, **lps[i]** is the length of longest proper prefix of **pat[0..i]** which is also a suffix of **pat[0..i]**.

Preprocessing Overview:

- We search for lps in subpatterns. More clearly we focus on sub-strings of patterns that are both prefix and suffix.
- For each sub-pattern pat[0..i] where i = 0 to m-1, lps[i] stores the length of the maximum matching proper prefix which is also a suffix of the sub-pattern pat[0..i].

> *lps[i] = the longest proper prefix of pat[0..i] which is also a suffix of pat[0..i].*

Example of lps[] construction:

Example 1: Pattern "aabaaac"

At index 0: "a" → No proper prefix/suffix → lps[0] = 0
At index 1: "aa" → "a" is both proper prefix and suffix →

lps[1] = 1
At index 2: "aab" → No prefix matches suffix → lps[2] = 0
At index 3: "aaba" → "a" is proper prefix and suffix → lps[3] = 1
At index 4: "aabaa" → "aa" is proper prefix and suffix → lps[4] = 2
At index 5: "aabaaa" → "aa" is proper prefix and suffix → lps[5] = 2
At index 6: "aabaaac" → Mismatch, so reset → lps[6] = 0
Final lps[]: [0, 1, 0, 1, 2, 2, 0]

Example 2: Pattern "abcdabca"

At index 0: lps[0] = 0
At index 1: lps[1] = 0
At index 2:lps[2] = 0
At index 3: lps[3] = 0 (no repetition in "abcd")
At index 4: lps[4] = 1 ("a" repeats)
At index 5: lps[5] = 2 ("ab" repeats)
At index 6: lps[6] = 3 ("abc" repeats)
At index 7: lps[7] = 1 (mismatch, fall back to "a")
Final LPS: [0, 0, 0, 0, 1, 2, 3, 1]

More Examples of lps[]

For the pattern “AAAA”, lps[] is [0, 1, 2, 3]

For the pattern “ABCDE”, lps[] is [0, 0, 0, 0, 0]

For the pattern “AABAACAABAA”, lps[] is [0, 1, 0, 1, 2, 0, 1, 2, 3, 4, 5]

For the pattern “AAACAAAAAC”, lps[] is [0, 1, 2, 0, 1, 2, 3, 3, 3, 4]

For the pattern “AAABAAA”, lps[] is [0, 1, 2, 0, 1, 2, 3]

Algorithm for Construction of LPS Array

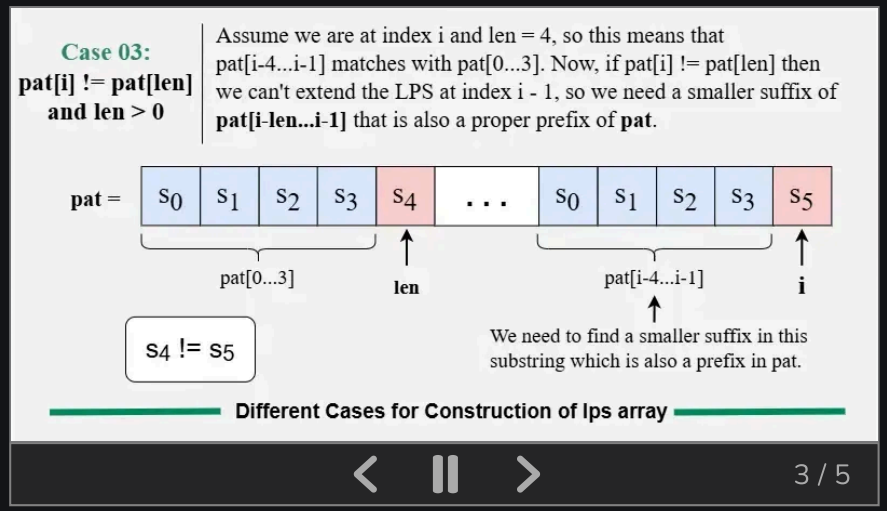
***lps[0]** is always **0** since a string of length one has no non-empty proper prefix. We store the value of the previous LPS in a variable **len**, initialized to 0. As we traverse the pattern, we compare the current character at index **i**, with the character at index **len**.*

***Case 1 - pat[i] = pat[len]:** this means that we can simply extend the LPS at the previous index, so increment len by 1 and store its value at lps[i].*

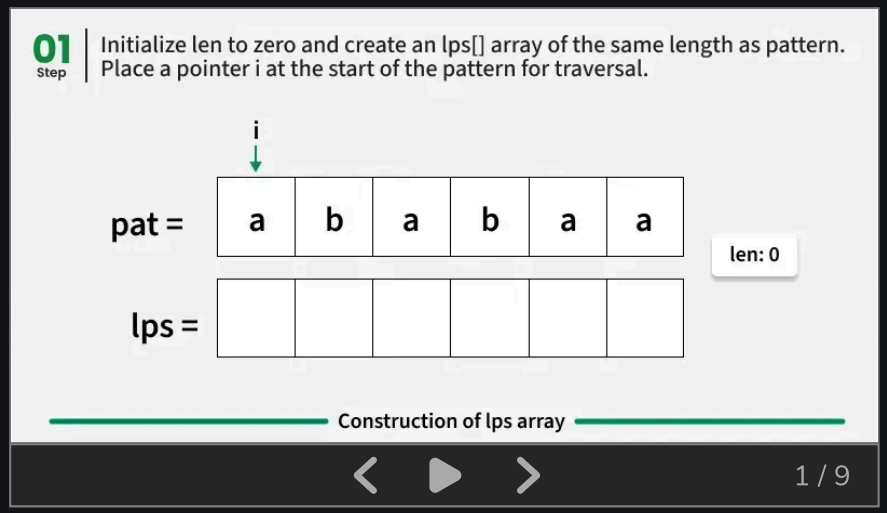
***Case 2 - pat[i] != pat[len] and len = 0:** it means that there were no matching characters earlier and the current characters are also not matching, so lps[i] = 0.*

***Case 3 - pat[i] != pat[len] and len > 0:** it means that we can't extend the LPS at index **i-1**. However, there may be a smaller prefix that matches the suffix ending at i. To find this, we look for a smaller suffix of **pat[i-len...i-1]** that is also a proper prefix of pat. We then attempt to match **pat[i]** with the next character of this prefix. If there is a match, **pat[i]** = length of that matching prefix. Since lps[i-1] equals len, we know that pat[0...len-1] is the same as pat[i-len...i-1]. Thus, rather than searching through pat[i-len...i-1], we can use lps[len - 1] to update len, since that part of the pattern has already been matched.*

Refer the below illustration for better explanation of all the cases:



Example of Construction of LPS Array:



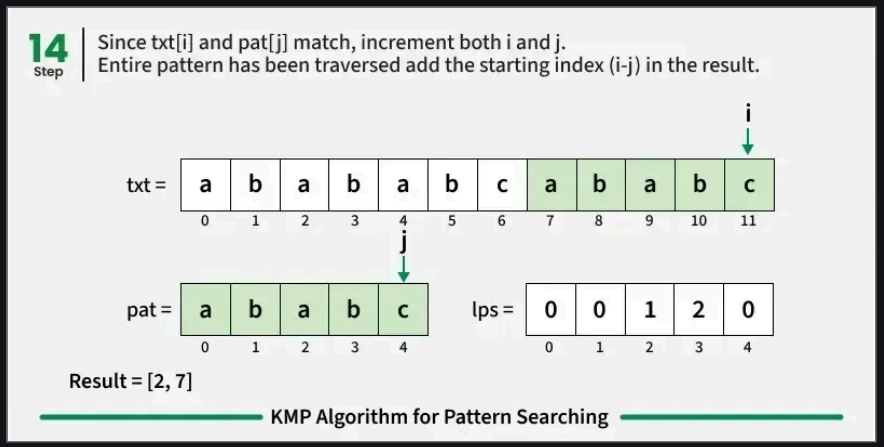
Implementation of KMP Algorithm

*We initialize two pointers, one for the text string and another for the pattern. When the characters at both pointers match, we **increment** both pointers and continue the comparison. If they do not match, we reset the pattern pointer to the **last value** from the LPS*

*array, because that portion of the pattern has already been **matched** with the text string. Similarly, if we have traversed the entire pattern string, we add the **starting** index of occurrence of pattern in text, to the result and continue the search from the lps value of **last** element of the pattern.*

Let's say we are at position i in the text string and position j in the pattern string when a mismatch occurs:

- At this point, we know that **pat[0..j-1]** has already matched with **txt[i-j..i-1]**.
- The value of **lps[j-1]** represents the length of the longest **proper prefix** of the substring pat[0..j-1] that is also a **suffix** of the same substring.
- From these two observations, we can conclude that there's no need to recheck the characters in **pat[0..lps[j-1]]**. Instead, we can directly resume our search from **lps[j-1]**.



C++ Java Python C# JavaScript

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

void constructLps(string &pat, vector<int> &lps)

    // len stores the length of longest prefix wh
    // is also a suffix for the previous index
    int len = 0;

    // lps[0] is always 0
    lps[0] = 0;

    int i = 1;
    while (i < pat.length()) {

        // If characters match, increment the siz
        if (pat[i] == pat[len]) {
            len++;
            lps[i] = len;
            i++;
        }

        // If there is a mismatch
        else {
            if (len != 0) {

                // Update len to the previous lps
```

```
                // to avoid reduntant comparisons
                len = lps[len - 1];
            }
            else {

                // If no matching prefix found, s
to 0
                lps[i] = 0;
                i++;
            }
        }
    }
}

vector<int> search(string &pat, string &txt) {
    int n = txt.length();
    int m = pat.length();

    vector<int> lps(m);
    vector<int> res;

    constructLps(pat, lps);

    // Pointers i and j, for traversing
    // the text and pattern
    int i = 0;
    int j = 0;

    while (i < n) {

        // If characters match, move both pointer
        if (txt[i] == pat[j]) {
            i++;
            j++;
```

```
            // If the entire pattern is matched
            // store the start index in result
            if (j == m) {
                res.push_back(i - j);

                // Use LPS of previous index to
                // skip unnecessary comparisons
                j = lps[j - 1];
            }
        }

        // If there is a mismatch
        else {

            // Use lps value of previous index
            // to avoid redundant comparisons
            if (j != 0)
                j = lps[j - 1];
            else
                i++;
        }
    }
    return res;
}

int main() {
    string txt = "aabaacaadaabaaba";
    string pat = "aaba";

    vector<int> res = search(pat, txt);
    for (int i = 0; i < res.size(); i++)
        cout << res[i] << " ";
```

```
    return 0;
}
```

Output

```
0 9 12
```

Time Complexity: O(n + m), where n is the length of the text and m is the length of the pattern. This is because creating the LPS (Longest Prefix Suffix) array takes O(m) time, and the search through the text takes O(n) time.
Auxiliary Space: O(m), as we need to store the LPS array of size m.

Real-Life Applications

- Text Editors (Find feature)
- Plagiarism Detection
- Bioinformatics (DNA sequence matching)
- Spam Detection Systems
- Search Engines

Related Problems

- Case Insensitive Search
- Find All Occurrences of Subarray in Array
- Minimum Characters to Add at Front for Palindrome
- Check if Strings Are Rotations of Each Other

- Minimum Repetitions of s1 such that s2 is a substring of it
- Longest prefix which is also suffix

Comment | K kartik | 649

Explore

- DSA Fundamentals
- Data Structures
- Algorithms
- Advanced
- Interview Preparation
- Courses